Airline Reservation System

A practical walkthrough: user input → stored data → results.

Python console application + SQLite database

Manage flights, passengers and tickets.

Book seats, search records, update details and cancel tickets.

Calculate revenue and occupancy. Export results.txt.

Three modules, one workflow

The menu collects input; booking rules validate it; SQLite stores it.

main.py

Batyrlan
Input and output
Menus and errors

→

airline.py

Alibek
Booking and search
Statistics and export

→

database.py

Batyr
Tables and SQL
Persistent data

Example: enter a seat → check availability → save a ticket → show its ID.

How the tables are connected

A ticket links one passenger to one flight.

flights

id [primary key]
route and departure
capacity and price

1:N

tickets

flight_id [foreign key]
passenger_id [foreign key]
seat and booking price

N:1

passengers

id [primary key]
name
unique email

One flight → many tickets. One passenger → tickets on different flights.

Database: protect booking relationships

database.py • VS Code screenshot

```
40          self.connection.execute(
41              """CREATE TABLE IF NOT EXISTS tickets (
42                  id INTEGER PRIMARY KEY,
43                  flight_id INTEGER NOT NULL REFERENCES flights(id),
44                  passenger_id INTEGER NOT NULL REFERENCES passengers(id),
45                  seat INTEGER NOT NULL CHECK (seat > 0),
46                  price INTEGER NOT NULL CHECK (price > 0),
47                  UNIQUE (flight_id, seat),
48                  UNIQUE (flight_id, passenger_id)
49              )"""
50          )
```

Foreign keys require an existing flight and passenger. UNIQUE prevents duplicate seats and duplicate passenger bookings on the same flight.

Initial data: created only once

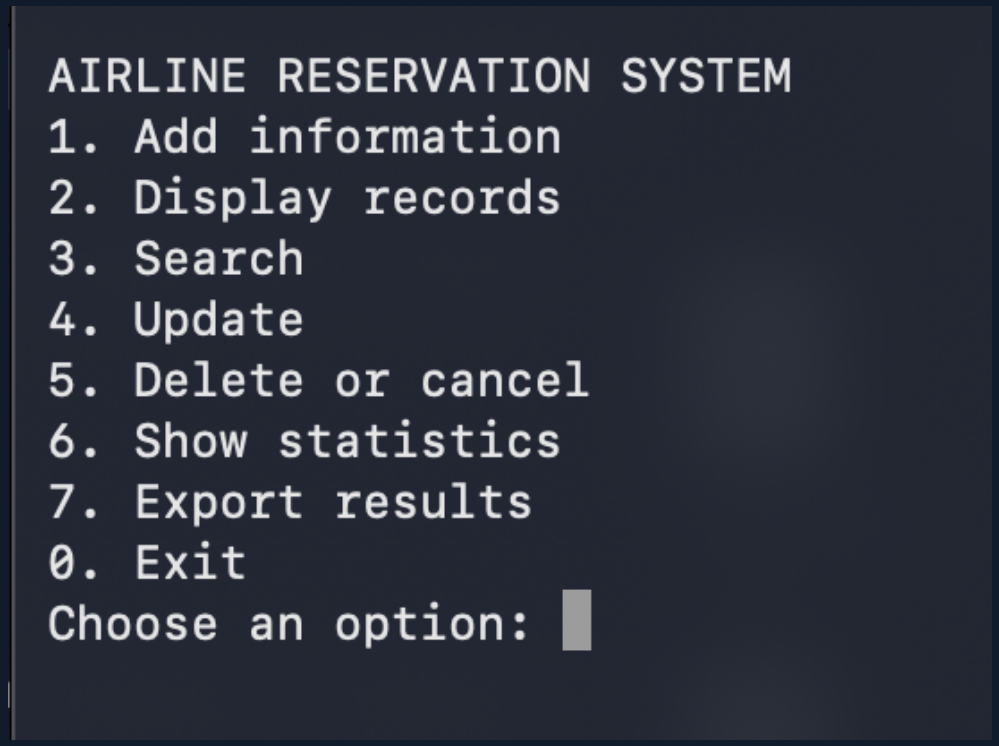
database.py • VS Code screenshot

```
61      def seed(self):
62          for table in ("flights", "passengers", "tickets"):
63              if self.fetch(f"SELECT COUNT(*) AS total FROM {table}")[0]["total"]:
64                  return False
65          generator = random.Random(26)
```

seed() stops if any table contains records. A fully empty database receives 20 flights, 50 passengers and 40 tickets; random uses a fixed seed.

Start with the main menu

Run: python3 main.py



Choose an operation from 1 to 7. The menu repeats after each action; 0 exits the program.

1. Add a flight

Result: flight 21 • KC900 • 90 seats • 45,000 KZT

```
Choose an option: 1
1. Flight
2. Passenger
3. Ticket
0. Back
Choose a record type: 1
Flight number (KC123): KC900
Origin: Almaty
Destination: Karaganda
Departure (YYYY-MM-DD HH:MM): 2026-11-25 14:30
Total seats: 90
Ticket price (whole KZT): 45000
Record saved. ID: 21
```

Add information → Flight. The user enters the route, departure, capacity and price. The program confirms the new flight ID.

Code: validate and save the flight

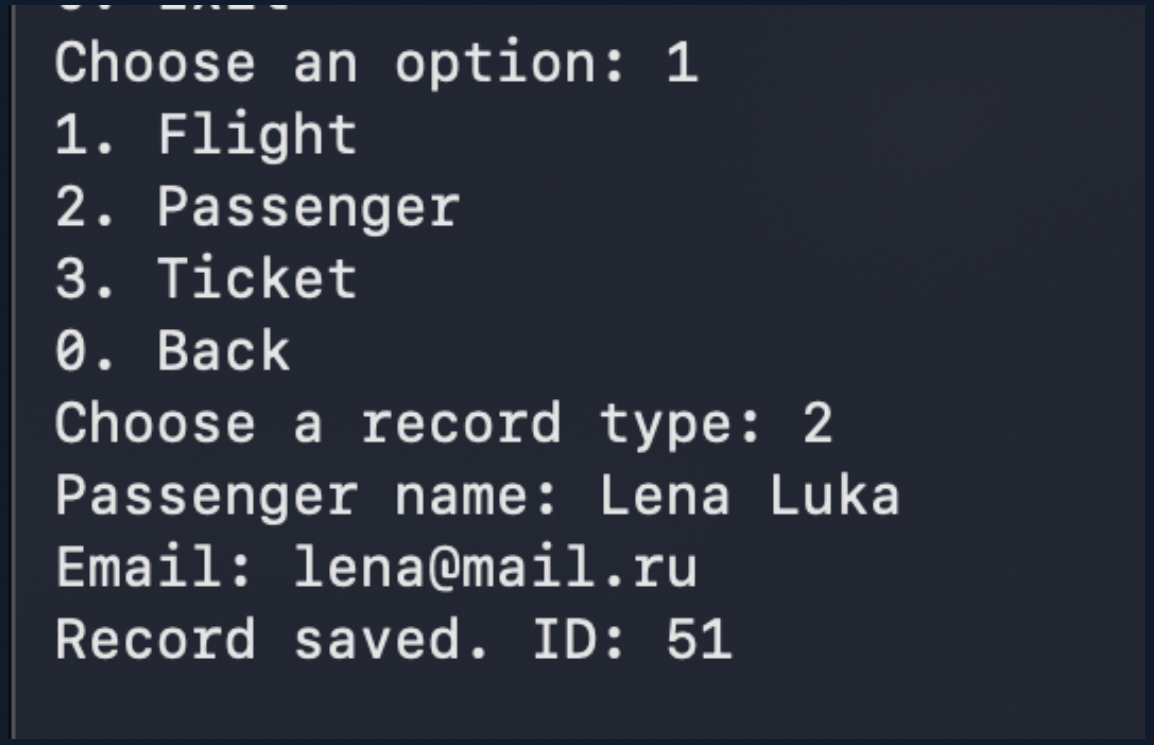
airline.py • VS Code screenshot

```
62      def add_flight(self, number, origin, destination, departure, seats, price):
63          values = self.flight_values(
64              number, origin, destination, departure, seats, price
65          )
66          return self.database.execute(
67              """INSERT INTO flights (number, origin, destination, departure, seats, price)
68                 VALUES (?, ?, ?, ?, ?, ?)""",
69              values,
70          )
71
```

add_flight() validates the supplied fields, then inserts them. The question marks receive values separately; execute() commits and returns the new ID.

2. Add a passenger

Result: passenger 51 • Lena Luka



Add information → Passenger. The name and email are saved as a passenger record. The generated ID is used when booking tickets.

Code: validate passenger details

airline.py • VS Code screenshot

```
54
55      def passenger_values(self, name, email):
56          name = self.required_text(name, "Name")
57          email = self.required_text(email, "Email").lower()
58          if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", email):
59              raise ValueError("Enter a valid email address, such as alex@example.com.")
60          return name, email
```

required_text() rejects empty input. Email is converted to lowercase; re.fullmatch() checks its format. add_passenger() then saves the validated values.

3. Book a ticket: open the form

Choose 1 — Add information, then 3 — Ticket.

STEP 1 / Select the operation

```
Choose an option: 1
1. Flight
2. Passenger
3. Ticket
0. Back
Choose a record type: 3
```

STEP 2 / Find the flight in the displayed table

```
 ID | NUMBER | ORIGIN | DESTINATION |    DEPARTURE     | SEATS | PRICE | BOOKED
----+--------+--------+-------------+------------------+-------+-------+-------
 21 | KC900  | Almaty | Karaganda   | 2026-11-25 14:30 | 90    | 45000 | 2
```

Flight KC900 has ID 21 → enter 21 at the Flight ID prompt.

3. Book a ticket: choose the passenger

Use the ID column, not the passenger name or the row position.

STEP 3 / Find the passenger in the displayed table

```
ID |      NAME      |           EMAIL
---+----------------+-------------------------
51 | Lena Nika      | lena12@gmail.ru
```

Passenger ID = 51

Next: enter Flight ID 21, Passenger ID 51 and an available seat.

The name is Lena Nika after the update; the passenger ID stays 51.

3. Book a ticket: save the booking

Result: ticket 43 • flight 21 • passenger 51 • seat 2

```
Flight ID: 21
Seat range: 1–90
Booked seats: 1
Passenger ID: 51
Seat number: 2
Record saved. ID: 43
```

Seat 1 is already booked. Passenger 51 selects available seat 2 on flight 21. The program saves the booking and returns ticket ID 43.

Code: check the seat before saving

airline.py • VS Code screenshot

```
77
78      def check_booking(self, flight_id, passenger_id, seat, ticket_id=0):
79          flight = self.get_record("flights", flight_id)
80          self.get_record("passengers", passenger_id)
81          self.positive_integer(seat, "Seat")
82          if seat > flight["seats"]:
83              raise ValueError(f"Seat must be between 1 and {flight['seats']}.")
84          matches = self.database.fetch(
85              """SELECT seat FROM tickets
86                 WHERE flight_id = ? AND id != ? AND (seat = ? OR passenger_id = ?)""",
87              (flight_id, ticket_id, seat, passenger_id),
88          )
89          if any(row["seat"] == seat for row in matches):
90              raise ValueError("This seat is already booked. Choose another seat.")
91          if matches:
92              raise ValueError("This passenger already has a ticket for this flight.")
93          return flight
94
```

check_booking() checks IDs, the seat range, occupied seats and repeat passenger bookings. A conflict raises ValueError; otherwise the flight is returned.

Code: save the ticket and its price

airline.py • VS Code screenshot

```
102     def add_ticket(self, flight_id, passenger_id, seat):
103         flight = self.check_booking(flight_id, passenger_id, seat)
104         return self.database.execute(
105             """INSERT INTO tickets (flight_id, passenger_id, seat, price)
106                VALUES (?, ?, ?, ?)""",
107             (flight_id, passenger_id, seat, flight["price"]),
108         )
```

add_ticket() runs the checks, then stores the flight, passenger, seat and current price.
Later changes to the flight price do not alter this ticket price.

4. Search for a passenger

Query: Lena → passenger 51, Lena Luka

```
Choose an option: 3
1. Passenger by name or ID
2. Flights by destination
0. Back
Choose a search: 1
Passenger name or ID: Lena
ID |     NAME    |     EMAIL
---+-------------+--------------
51 | Lena Luka | lena@mail.ru
```

Search → Passenger by name or ID. A partial name is enough; matching is case-insensitive.
The result displays ID, name and email.

5. Search flights by destination

Query: Karaganda → flight KC900, one booked seat

```
Choose an option: 3
1. Passenger by name or ID
2. Flights by destination
0. Back
Choose a search: 2
Destination: Karaganda
ID | NUMBER | ORIGIN | DESTINATION |    DEPARTURE     | SEATS | PRICE | BOOKED
---+--------+--------+-------------+------------------+-------+-------+-------
21 | KC900  | Almaty | Karaganda   | 2026-11-25 14:30 | 90    | 45000 | 1
```

Search → Flights by destination. The result shows the route, departure, capacity, price and number of booked tickets.

Code: search and handle missing records

Passenger search uses SQL; the display function handles an empty result.

```
130     def search_passengers(self, query):
131         query = self.required_text(query, "Search")
132         return self.database.fetch(
133             """SELECT * FROM passengers
134                WHERE CAST(id AS TEXT) = ? OR instr(casefold(name), casefold(?)) > 0
135                ORDER BY id""",
136             (query, query),
137         )
```

```
Choose an option: 3
1. Passenger by name or ID
2. Flights by destination
0. Back
Choose a search: 1
Passenger name or ID: Somae
No records found.
```

CAST compares ID as text.
casefold ignores letter case.
instr finds part of a name.
Empty rows → No records found.

6. Update a passenger

Passenger 51: Lena Luka → Lena Nika

```
Choose an option: 4
1. Flight
2. Passenger
3. Ticket
0. Back
Choose a record type: 2
Record ID to update: 51
ID |    NAME    |    EMAIL
---+------------+-------------
51 | Lena Luka | lena@mail.ru
Enter all replacement fields.
Passenger name: Lena Nika
Email: lena12@gmail.ru
Record updated.
```

Update → Passenger. The program shows the old record, accepts replacement fields and confirms the update. airline.py: update_passenger() runs UPDATE.

7. Cancel a ticket

Result: ticket 41 deleted after confirmation

```
Choose an option: 5
1. Flight
2. Passenger
3. Ticket
0. Back
Choose a record type: 3
Record ID to delete or cancel: 41
ID | FLIGHT_ID | PASSENGER_ID | SEAT | PRICE
---+-----------+--------------+------+------
41 | 1         | 14           | 3    | 45000
Type yes to confirm: yes
Record deleted or ticket cancelled.
```

Delete or cancel → Ticket → ID 41 → yes. This is a separate ticket on flight 1, not ticket 43 from the booking example. delete_record() removes it.

8. Calculate results and export

Recorded session: statistics before cancellation, then export through option 7.

```
Choose an option: 6
AIRLINE RESERVATION SYSTEM
Flights: 21
Passengers: 51
Active tickets: 42
Total seats: 1601
Total ticket revenue: 2494000 KZT
Seat occupancy rate: 2.62%
```

Revenue = SUM(ticket prices)

2,494,000 KZT

Occupancy = 42 / 1601 × 100

2.62%

```
Choose an option: 7
Report saved: /Users/albkqw/Desktop/python/airlines/results.txt
```

statistics() calculates metrics; export_results() saves records and results.txt.

Our contributions and what we learned

Two classes • three related tables • CRUD • search • calculations • file export

Kushekbayev Batyrlan	**main.py** Menu, input, tables and error messages
Aben Batyr	**database.py** SQLite, relationships and random data
Abduldinov Alibek	**airline.py** Booking, re validation, search and results

Next: live demonstration and questions.